MINISTRY OF EDUCATION AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers Informatics and Microelectronics

Department of Software Engineering and Automation

Report on laboratory work 4

The xv6 shell from the outside

Course Operating Systems

Done by: st. gr. FAF-242, Tacu Igor
Checked by: Reițman Patricia

Chișinău, 2026

Purpose and execution record

All twelve figures are native Terminal screenshots supplied by the author from direct command execution and source inspection on the Ubuntu server on 9 November 2026. The xv6 tests use an isolated source copy and its generated test disk. The directory d already existed during the directory-handling capture; grep READ limits the displayed listing. Bash comparisons use separate temporary directories. Source files and earlier full execution transcripts accompany the screenshots. The submission was prepared with coding assistance; MIT xv6 is identified as upstream in the source provenance file.

Part 1 Sleep and the current interface

The header declares pause rather than sleep. The submitted utility calls pause with validated decimal ticks; sleep 20 returns normally, missing input produces usage, and negative input produces a diagnostic. The Makefile contains the _sleep entry. Forked processes inherit the binary and descriptor context, while pause is the kernel operation that performs the delay.

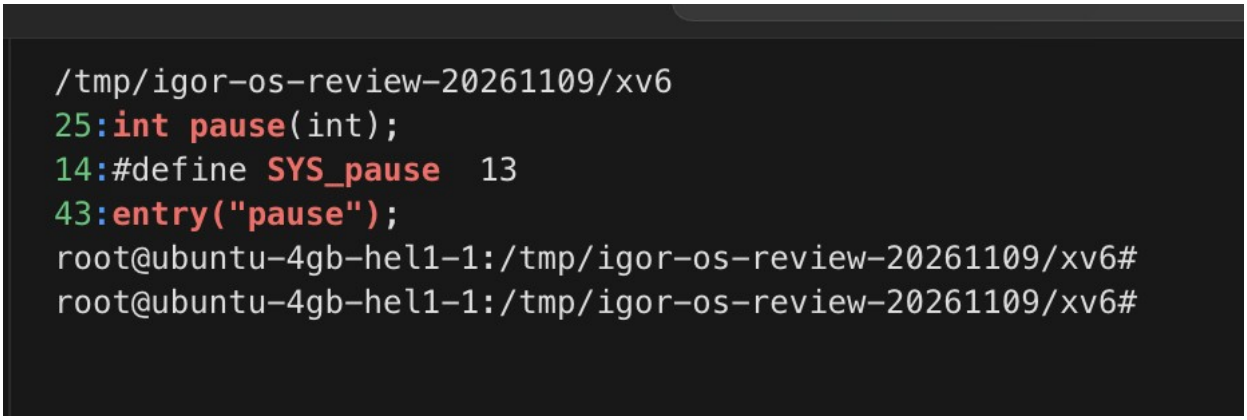


Figure 1.1 – The pause declaration

```
 1  #include "kernel/types.h"
 2  #include "user/user.h"
 3
 4  int
 5  main(int argc, char **argv)
 6  {
 7    if(argc != 2 || argv[1][0] == 0){
 8      fprintf(2, "usage: sleep nonnegative_ticks\n");
 9      exit(1);
10    }
11    int ticks = 0;
12    for(char *p = argv[1]; *p; p++){
13      if(*p < '0' || *p > '9' || ticks > (2147483647 - (*p - '0')) / 10){
14        fprintf(2, "sleep: invalid tick count\n");
15        exit(1);
16      }
17      ticks = ticks * 10 + (*p - '0');
18    }
19    exit(pause(ticks) < 0);
20  }
138:    $U/_sleep\
root@ubuntu-4gb-hel1-1:/tmp/igor-os-review-20261109/xv6#
```

Figure 1.2 – Sleep source and registration

```
xv6 kernel is booting

init: starting sh
$ sleep 20
$ echo finished
finished
$ sleep
usage: sleep nonnegative_ticks
$ sleep -1
sleep: invalid tick count
$
```

Figure 1.3 – Sleep with valid and invalid arguments

Part 2 Comparing every command in the two shells

The Bash test starts in a fresh directory containing README. Both shells provide execution, descriptor redirection, pipelines, command lists, grouping and background commands. Bash

additionally expands wildcards and variables, removes syntactic quotes, and offers readline history and completion. The table records the actual small-shell behavior rather than assuming it implements Bash syntax.

Command	Bash	xv6	Reason
echo hello > f ; cat < f	hello	hello	descriptor redirection
ls | grep READ	README	README line	pipe
(echo a; echo b) | wc	2 lines 2 words 4 bytes	2 2 4	group and list
echo one ; echo two	one then two	one then two	sequence
echo bg &	bg and prompt	bg and prompt	background
ls *	expands existing names	cannot open *	Bash pathname expansion
echo $HOME	/root	$HOME	Bash variable expansion
echo "a b"	a b	"a b"	Bash quote removal
mkdir d ; cd d ; ls ..	lists parent after changing directory	exec cd failed; lists from original directory	xv6 handles cd only as a whole input line
Up Tab Ctrl+R	history and completion	not implemented as Bash readline features	source and PTY inspection

There is an important distinction in the last directory case. A cd embedded in a semicolon list is passed through the xv6 execution parser and fails as an external command. Issuing cd d as its own input line does change the parent shell. From there a bare ls fails because xv6 has no PATH search, while /ls .. succeeds. For interactive work, history and completion are valuable additions. Basic quote handling would be a localized tokenizer improvement; full pathname expansion would also require directory traversal.

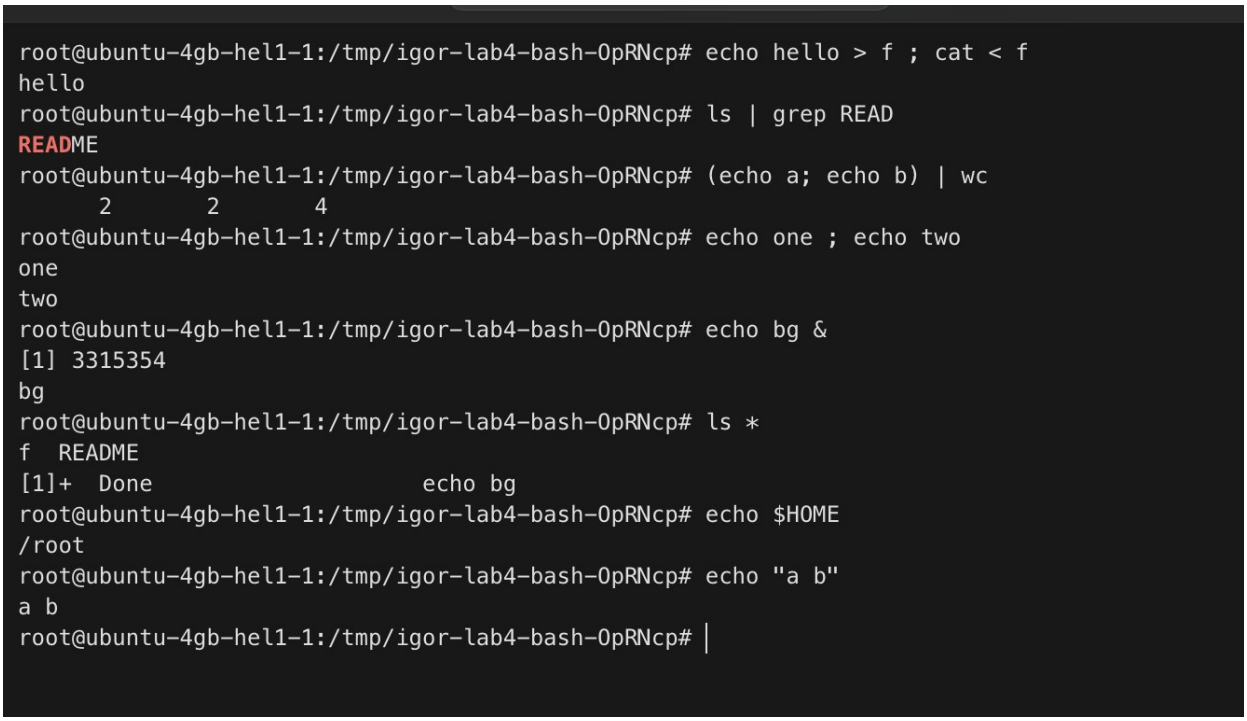


Figure 2.1 – The original comparison commands in Bash

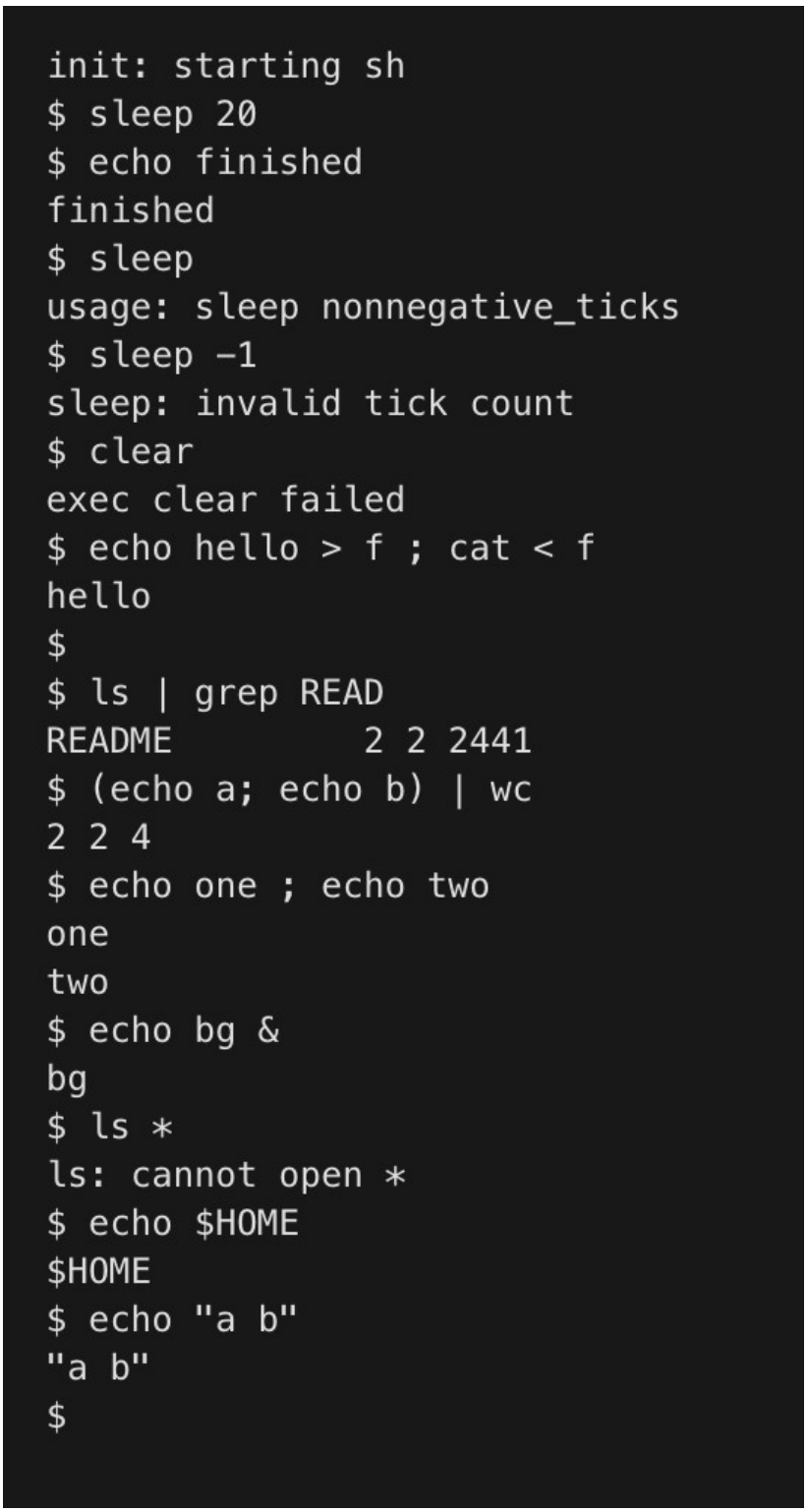


Figure 2.2 – The same comparison commands in xv6



Figure 2.3 – Directory handling and absolute ls (README-filtered listing)

Part 3 Who expands the argument vector

Both inspectors print only argc and argv. The Linux wildcard experiment in this additional test directory had four C files and one quoted x y operand, giving argc 6 including argv zero. Xv6 passed four arguments: args, the literal *.c, and two values containing the double quote marks. The shell, not the inspector or kernel, is responsible for expansion and syntactic quote removal. Repeated unquoted spaces delimit arguments in both shells; they do not become empty arguments.

```
root@ubuntu-4gb-hel1-1:/tmp/igor-os-review-20261109/xv6# cat -n user/args.c
     1  #include "kernel/types.h"
     2  #include "user/user.h"
     3
     4  int
     5  main(int argc, char **argv)
     6  {
     7    printf("argc=%d\n", argc);
     8    for(int i = 0; i < argc; i++) printf("argv[%d]=[%s]\n", i, argv[i]);
     9    exit(0);
    10  }
root@ubuntu-4gb-hel1-1:/tmp/igor-os-review-20261109/xv6# grep -n '_args' Makefile
139:    $U/_args\
root@ubuntu-4gb-hel1-1:/tmp/igor-os-review-20261109/xv6#
```

Figure 3.1 – Args source and UPROGS entry

```
$ args one    two three
argc=4
argv[0]=[args]
argv[1]=[one]
argv[2]=[two]
argv[3]=[three]
$
$ args *.c "x y"
argc=4
argv[0]=[args]
argv[1]=[*.c]
argv[2]=["x]
argv[3]=[y"]
$ args $HOME
argc=2
argv[0]=[args]
argv[1]=[$HOME]
$ args 'single quoted'
argc=3
argv[0]=[args]
argv[1]=['single]
argv[2]=[quoted']
$
```

Figure 3.2 – Argument arrays from xv6

```
root@ubuntu-4gb-hel1-1:/tmp/igor-lab4-args-K3nh8o# ./args one    two three
argc=4
argv[0]=[./args]
argv[1]=[one]
argv[2]=[two]
argv[3]=[three]
root@ubuntu-4gb-hel1-1:/tmp/igor-lab4-args-K3nh8o# ./args *.c "x y"
argc=6
argv[0]=[./args]
argv[1]=[alpha.c]
argv[2]=[args_linux.c]
argv[3]=[beta.c]
argv[4]=[sample.c]
argv[5]=[x y]
root@ubuntu-4gb-hel1-1:/tmp/igor-lab4-args-K3nh8o# ./args $HOME
argc=2
argv[0]=[./args]
argv[1]=[/root]
root@ubuntu-4gb-hel1-1:/tmp/igor-lab4-args-K3nh8o# ./args '$HOME' "$HOME"
argc=3
argv[0]=[./args]
argv[1]=[$HOME]
argv[2]=[/root]
root@ubuntu-4gb-hel1-1:/tmp/igor-lab4-args-K3nh8o# ./args 'single quoted'
argc=2
argv[0]=[./args]
argv[1]=[single quoted]
root@ubuntu-4gb-hel1-1:/tmp/igor-lab4-args-K3nh8o#
```

Figure 3.3 – Argument arrays from Linux

Part 4 Reading the shell

This version of user/sh.c has 499 lines. EXEC represents a plain command; REDIR is introduced by < or >; PIPE by |; LIST by a semicolon; and BACK by ampersand. Parentheses group a parsed command tree. Cd is recognized in the parent loop before forking and parsing a general command, so its directory change persists. The parser has no Bash parameter-expansion or globbing stage, explaining the literal * and $HOME results. The five command kinds are visible both in their definitions and in runcmd dispatch.

```
root@ubuntu-4gb-hel1-1:/tmp/igor-lab4-args-K3nh8o# cd /tmp/igor-os-review-20261109/xv6
root@ubuntu-4gb-hel1-1:/tmp/igor-os-review-20261109/xv6# grep -nE '^#define (EXEC|REDIR|PIPE|LIST|BACK)' u
ser/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
root@ubuntu-4gb-hel1-1:/tmp/igor-os-review-20261109/xv6#
```

Figure 4.1 – Defined command kinds

```
root@ubuntu-4gb-hel1-1:/tmp/igor-os-review-20261109/xv6# grep -nE 'case (EXEC|REDIR|PIPE|LIST|BACK)' user/
sh.c
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
469:  case EXEC:
475:  case REDIR:
481:  case PIPE:
487:  case LIST:
493:  case BACK:
root@ubuntu-4gb-hel1-1:/tmp/igor-os-review-20261109/xv6# \
```

Figure 4.2 – Dispatch cases

```
root@ubuntu-4gb-hel1-1:/tmp/igor-os-review-20261109/xv6# sed -n '159,177p' user/sh.c
  // Read and run input commands.
  while (getcmd(buf, sizeof(buf)) >= 0) {
    char *cmd = buf;
    while (*cmd == ' ' || *cmd == '\t')
      cmd++;
    if (*cmd == '\n') // is a blank command
      continue;
    if (cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' ') {
      // Chdir must be called by the parent, not the child.
      cmd[strlen(cmd) - 1] = 0; // chop \n
      if (chdir(cmd + 3) < 0)
        fprintf(2, "cannot cd %s\n", cmd + 3);
    } else {
      if (fork1() == 0)
        runcmd(parsecmd(cmd));
      wait(0);
    }
  }
  exit(0);
root@ubuntu-4gb-hel1-1:/tmp/igor-os-review-20261109/xv6#
```

Figure 4.3 – Parent shell implementation of cd

Conclusions

The two shells share process and descriptor mechanisms but differ before exec in parsing and expansion. The argument inspector isolates those differences. The additional cd-list test also shows why a small shell feature cannot be assumed to compose exactly as in Bash. Both requested programs and the integrated Makefile are submitted with the full xv6 source and raw console evidence.